Pointers

Question: If those memory addresses are so fundamental for programming, can they be represented in the C language? The answer is YES - addresses can be stored inside special variables called *pointers*. Let's see an example:

```
int *p_int;
```

p_int is a pointer to integers. In other words, p_int is a variable that can hold **memory addresses** of integer variables. If so, p_int can hold the address of the counter variable:

```
p_int = &counter;
```

It is also useful to get the **value** stored in the memory location pointed by p_int. This is call pointer dereferencing. We use the indirection operator (*) as the prefix to dereference a pointer:

```
*(pointer_name)
```

Therefore the while loop can become:

```
while (*(p_int) < 10) {
   (*p_int) ++;
}
```

(p_int)* means the **value at the **address** currently stored in the p_int poiner, which, in this case, is the value of the counter variable.

Now lets add *p_int to the Expression tab and look at its value. Indeed *p_int is an alias for counter.

Finally lets see the incredible power we have with pointers:

```
p_int = (int *)0x20000010;
*p_int = 0xDEADBEEF;
```

Examine the value stored in 0x20000010 in the Memory tab (you can also try with a misaligned address i.e. 0x20000012)

Exercise 1:

In C, create an integer variable x with an initial value of 10. Use a pointer to reference and change the value of x to 20. Dereference the pointer to show the new value of x.

Exercise 2:

Given the integer array int arr[3] = {5, 10, 15}; in C:
Use pointer arithmetic to calculate the sum of the first and second elements.
Store the result in a new variable sum.

Exercise 3:

In C, define an array int nums[4] = {2, 4, 6, 8};.
Use a pointer to iterate through the array and multiply each element by 2.

Exercise 4:
In C, write a function void swap(int *a, int *b) to swap two integers using call by reference.
Test the function with x = 7 and y = 9.

ARM ISA
Exercise 1: Simple arithmetic
Set R1 = 0x30 and R2 = 0x1A. Then write ARM assembly code to compute the following:

- R3 = (R1 + R2) × 4
- R4 = R1 $\oplus$ R2
- R5 = R2 >> 3
- R6 = R1 × 8 (using only shifts)

What is the result?

Exercise 2: Data movement
Write ARM assembly code to:

- Load the value 0x1234 into r0.
- Store the value in memory at address 0x20000000.
- Load it back from memory into r1.
- Modify the code to increment the value in memory by 1

Exercise 3: Simple arithmetic
Store the following array of integers in memory starting at address 0x20000000: [0x9, 0x25, 0x46].
Then, for this array perform the following steps:

1. Write ARM assembly code to calculate the sum of the first three elements.
2. Multiply the second element by 3 and store it back in memory.